

Steel Fab Enterprises (SFE) Milestone Tracking & Estimation System

Complete Operational Workflow & Technical Reference Manual (A-Z)

1. Application Overview

The Steel Fab Enterprises (SFE) Milestone Tracking & Estimation System is an enterprise resource planning (ERP) portal designed specifically for steel fabrication and erection workflows. It coordinates the lifecycle of commercial steel construction projects from the initial Request for Quotation (RFQ) through estimation, bidding, sequencing, detailing, shop fabrication, shipping, erection, and final handover.

Tech Stack Architecture

- * **Frontend**: React (Vite-powered Single Page Application), styled with Tailwind CSS, utilizing Lucide icons for UI indicators.
- * **Backend**: Django REST Framework (DRF) serving JSON APIs, using a custom middleware system for request logging.
- * **Database**: SQLite for development environments (`db.sqlite3` / `vps_db.sqlite3`) and PostgreSQL for production environments.
- * **Offline Chatbot**: Python-based PDF parser (`pypdf`), local `BM25Okapi` search indexing, and a local Ollama API server running `llama3.2` models.

2. User Roles & Permissions

The application enforces role-based access control (RBAC) defined in the custom User database model (`accounts.User`). The system supports six user roles:

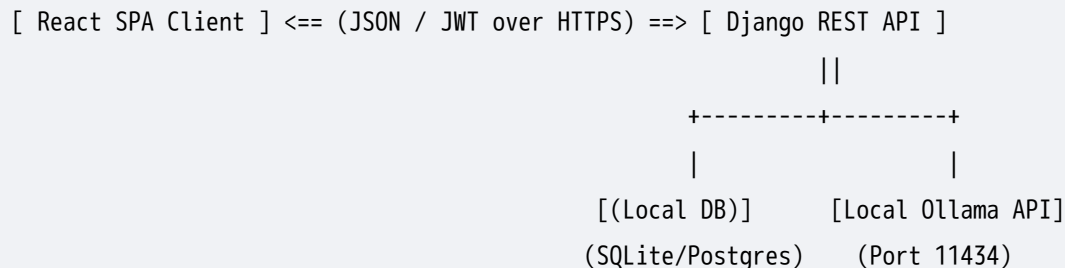
Role	Database Value	Description	Permissions
---	---	---	---
Admin	<code>admin</code>	System Administrator	Full access to all screens, read/write APIs, announcements, and context training.
Manager	<code>manager</code>	Operations Manager	Access to scheduling, capacity config, and approvals. Cannot change system settings.
Estimator	<code>estimator</code>	Bid Estimator	Read/write access to RFQ logging, estimations, bid calendars, and results.
Detailing	<code>detailing</code>	Detailing Partner	Access to Plan Creation and Plan Tracking. Read-only on estimation pricing.
Employee	<code>employee</code>	Shop Floor Staff	Read-only access to schedules. Can update item cutting/assembly logs.
Read-Only	<code>readonly</code>	Audit Guest	Read-only access to all dashboards and schedules.

No modification rights. |

3. System Architecture Summary

The SFE portal uses a decoupled client-server architecture:

```



```

* **API Gateway Proxy**: In development, Vite proxies requests prefixed with `/api` and `/SFE-media` directly to the Django server running at `http://localhost:8000`. In production, Nginx handles proxying to Gunicorn/Django.

* **Authentication Mechanism**: Stateless JWT authentication. Access tokens are stored in the client's `sessionStorage` and sent in the HTTP `Authorization` headers as `Bearer <token>`.

4. Authentication Workflow

User Sign Up & Registration

* **API Endpoint**: `POST /api/auth/register/`

* **Payload**: `{"email", "password", "first_name", "last_name", "role", "phone", "department"}`

* **Backend processing**:

1. Validates that the email is unique in the custom `users` database table.
2. Hashes the password using Django's default PBKDF2 hasher.
3. Saves a new user record with `role` defaulting to `'employee'` if unspecified.

User Login (Sign In)

* **API Endpoint**: `POST /api/auth/login/`

* **Payload**: `{"email", "password"}`

* **Backend processing**:

1. Authenticates against the user store using Django's standard backend.
2. Generates an Access Token (valid for 1 day) and Refresh Token (valid for 7 days) via

SimpleJWT.

3. Returns user credentials `{ "id", "email", "first\_name", "last\_name", "role"}` along with the token.

* **Frontend Action**: Saves the JWT token to `sessionStorage.setItem('token', token)` and user metadata to `sessionStorage.setItem('user', JSON.stringify(user))`. Redirects the router to `/dashboard`.

Password Reset Flow

* **API Endpoint**: `POST /api/auth/forgot-password/` -> Initiates request and generates a temporary OTP.

* **API Endpoint**: `POST /api/auth/reset-password/` -> Verifies OTP and commits new hashed password.

5. Module-by-Module Workflows

Module: Dashboard Overview (`/dashboard`)

Provides operational statistics and alerts.

* **UI Components**: Summary cards (Total Projects, In Progress, Yet to Start, Completed), Gantt charts for fabrication schedules, and announcements banner.

* **API Endpoints**: `GET /api/dashboard/stats/`

UI Action → Result Mapping

UI Element	User Action	Frontend Event	Backend API Call	DB Reads/Writes & Logic	Frontend Result
------------	-------------	----------------	------------------	-------------------------	-----------------

	:---	:---	:---	:---	:---
--	------	------	------	------	------

Gantt Chart Timeline	Hover over schedule bar	Displays tooltip	None (local state)		
	Displays timeline detail overlay				

Announcements Panel	Load Page	Fetch announcements	`GET /api/dashboard/announcements/`	Reads `Announcement` table where `from_date <= today` and `to_date >= today`	Renders announcement list
----------------------------	-----------	---------------------	-------------------------------------	--	---------------------------

New Announcement	Click "Create"	Opens Modal Form	`POST /api/dashboard/announcements/`	Writes new announcement record associated with current user	Refresh announcements grid
-------------------------	----------------	------------------	--------------------------------------	---	----------------------------

Module: RFQ Master (`/rfq`)

Logs Requests for Quotation and parses estimation scopes.

```
*  **UI Components**: Master spreadsheet grid, search filters, and estimators assignment panel.
*  **Database Models**: `apps.rfq.models.RFQMaster`
*  **API Endpoints**: `GET/POST /api/rfq/rfq/`
```

UI Action → Result Mapping

```
| UI Element | User Action | Frontend Event | Backend API Call | DB Reads/Writes & Logic |
Frontend Result |
| :--- | :--- | :--- | :--- | :--- | :--- |
| **Add RFQ Button** | Click button | Opens New RFQ Form Modal, queries next quote number | `GET /api/rfq/next-quote-no/` | Auto-calculates next serial Quote Number based on date prefix | Displays New RFQ Modal with pre-filled Quote Number |
| **New RFQ Modal Form** | Fill fields: **Quote No** (required), **Type** (Budget/Final/Rebid dropdown), **Quote Date** (date), **Bid Reference** (text), **Project Name** (required), **Project Comments** (text), **Bid Due Date/Time** (date/time), **Location** (text), **Distance (miles)** (number), select **Customer**, select **Decision to Bid** (Yes/No/NoBid/Bid), select **Scope of Work** (Detailing/Fabrication/Erection), select **Primary Estimator** & click "Create RFQ" | Validates required fields, sends payload | `POST /api/rfq/rfq/` | Writes validation parameters and logs record to `RFQMaster` table. | Closes modal, refreshes RFQ list, shows success toast. |
| **Delete RFQ Icon** | Click Trash icon | Prompts confirmation | `DELETE /api/rfq/rfq/<id>/` | Sets `deleted_at = timezone.now()` (Soft Delete) | Removes row from grid |
```

Module: Estimation & Erection Takeoff Layout (`/estimation-erection`)
Multi-tab engineering workbench to calculate project tonnage and material rates.

...

+-----+				
	Core		Breakdown	
+-----+				
	Tonnage, Crane Rental, Rigging Rates, and Joint Weights			
+-----+				

...

```
*  **Database Models**: `apps.erection_takeoff.models.ErectionTakeoff`,
`apps.field_moment_conn.models.ConnectionData`, `apps.misc_metals.models.MiscMetalsData`,
`apps.estimate_data.models.EstimateRate`
```

UI Action → Result Mapping

```
| UI Element / Tab | User Action | Frontend Event | Backend API Call | DB Reads/Writes & Logic
| Frontend Result |
| :--- | :--- | :--- | :--- | :--- | :--- |
| **Core Tab (Save Parameters)** | Click "Save Core" | Validate spans | `POST
/api/projects/<id>/estimate-data/core/` | Writes base dimensions to `CoreEstimation` |
Displays updated total base weights |
| **Erection Tab (Recalculate)** | Click "Recompute Erection" | Computes crew rates | `POST
/api/projects/<id>/erection-takeoff/calculate/` | Reads equipment rates, computes
`crane_rental_cost + rigging_hours * rate` | Updates price grid, shows total cost |
| **Field Moment Connection (FMC)** | Enter bolt count and save | Validates size | `POST
/api/projects/<id>/fmc/save/` | Writes connection parameters to `ConnectionData` | Calculates
connection hours and weight additions |
| **Rates Grid (Save Rates)** | Edit rates inline & click Save | Validates numbers | `PUT
/api/projects/<id>/estimate-data/rates/` | Writes updated rate matrices to `EstimateRate` |
Commits changes, highlights fields green |
```

— — —

```
### Module: Milestone Management (`/milestones`)
```

Tracks progress deliverables and deadlines.

```
*  **Database Models**: `milestones.models.Milestone`
```

```
*  **API Endpoints**: `/api/milestones/`
```

UI Action → Result Mapping

UI Element	User Action	Frontend Event	Backend API Call	DB Reads/Writes & Logic	
Frontend Result					
:---	:---	:---	:---	:---	:---
Status Filter Tab	Click "Overdue"	Filters local array	None	None	Filters table to display overdue records
Add Milestone Form	Click "Create"	Sends form data	`POST /api/milestones/`	Writes `Milestone` record with status `Pending`	Appends milestone to table
Mark Completed Checkbox	Click Checkbox	Sends patch payload	`PATCH /api/milestones/<id>/`	Writes `status = 'Completed'` to `Milestone`	Updates status badge to green

— — —

```
### Module: Project Master (projects)
```

Central log of commercial construction projects, scheduling sequences, and drawing releases.

- * **Database Models**: `projects.models.Project`, `projects.models.StructuralScheduleItem`
- * **API Endpoints**: `/api/projects/` (GET/POST/PUT/PATCH/DELETE)

UI Action → Result Mapping

UI Element	User Action	Frontend Event	Backend API Call	DB Reads/Writes & Logic	Frontend Result
	:---	:---	:---	:---	:---
Add Project Button	Click button	Resets form, opens project creation modal	None	None	Displays Project Modal dialog starting on "Basic Info" tab.
Project Form (Basic Info)	Input fields: Project Name (required), Project Code/Job Number (required), select Customer Name , select Detailer Vendor , select Project Manager Name , Total Tons (calculated weight), Total Manhours (estimated labor), Erection Date (date), select Status (Yet to Start/In Progress/Completed/On Hold/Delayed), select Priority (Low/Medium/High), plant Name , select Field Measure Required (Yes/No) & click Save	Validates inputs, sends project metadata	`POST /api/projects/`	Writes project configurations to the `Project` database table. Saves project details, auto-syncs erection dates to detailing sequences.	
Project Schedule Sequence Grid	Add sequence rows and input details: Seq # , Tons , Item Description , Category , Scheduled OFA , Actual OFA , Scheduled BFA , Actual BFA , RTS Date , plant Lead Time (Weeks) , Detailer/Vendor , Dwg Status , Notes	Auto-calculates budget plant hours (tons * average manhour rate)	`POST /projects/structural-schedules/`	Commits sequences linked to project ID to `StructuralScheduleItem` table. Renders interactive Gantt timelines for drawing release scheduling.	
Delete Project Icon	Click Delete	Prompts delete alert	`DELETE /api/projects/<id>/`	Deletes the `Project` record. Reloads Project list.	

Module: Production Schedule & Priority (`/production`)

Prioritizes and routes shop pieces (Plate, Angle, Structural).

- * **Database Models**: `production.models.ProductionSchedule`, `production.models.ProductionItem`, `production.models.ProductionPriorityItem`
- * **API Endpoints**: `/api/production/schedules/`, `/api/production/priorities/`

UI Action → Result Mapping

UI Element	User Action	Frontend Event	Backend API Call	DB Reads/Writes & Logic
------------	-------------	----------------	------------------	-------------------------

```

Frontend Result |
| :--- | :--- | :--- | :--- | :--- | :--- |
| **Priority Row Drag/Drop** | Reorder row | Changes priority index | `POST
/api/production/priorities/reorder/` | Updates `priority_index` on `ProductionPriorityItem` |
Re-sorts schedule table |
| **Process Routing Dropdown** | Select process step | Validates capacity | `PATCH
/api/production/items/<id>/` | Writes active process status (e.g. "Cutting") | Updates status
color on the grid |

---

```

Module: Capacity Mapping (`/production/capacity-mapping`)

Configures shop capacities, shift workers, and machines.

```

* **Database Models**: `production.models.Capacity`, `production.models.Machine`,
`production.models.Manpower`
* **API Endpoints**:
  * Capacity View: `/api/production/capacity/` (GET/POST/PUT/DELETE)
  * Machine View: `/api/production/machines/` (GET/POST/PUT/DELETE)
  * Manpower View: `/api/production/manpower/` (GET/POST/PUT/DELETE)

```

UI Action → Result Mapping

```

| UI Element | User Action | Frontend Event | Backend API Call | DB Reads/Writes & Logic |
Frontend Result |
| :--- | :--- | :--- | :--- | :--- | :--- |
| **Add Capacity Button** | Click button | Opens New Capacity Modal | None | None | Renders
capacity input form fields. |
| **Capacity Modal Form** | Fill fields: select **Shop Name**, **Location**, select
**Category** (Machine/Manual), select **Machine** (optional, filtered by shop), **Process
Name** (text), select **Months** checklist (multiple months), **Capacity/Day**,
**Capacity/Month**, **Capacity/Year** & click Save | Auto-calculates month/year values
dynamically when day rate is entered | `POST /api/production/capacity/` | Writes monthly
capacity profiles to `Capacity` table. | Closes modal, refreshes Capacity list. |
| **Add Machine Button** | Click button | Opens Machine Form Modal | None | None | Renders
machine registration fields. |
| **Machine Modal Form** | Fill fields: **Machine Name** (required), **Machine ID** (text),
**Make** (required), select **Shop** (or select "+ Add New Shop" and enter custom shop number
& name), **Model No**, **Serial No**, **Commissioned Date**, **Validity (MM/YY)**, and custom
key-value metadata rows & click Save | Compiles custom fields into a JSON string payload |
`POST /api/production/machines/` | Writes machine credentials and JSON other_fields to
`Machine` table. | Closes modal, refreshes Machine list. |

```

| ****Add Manpower Button**** | Click button | Opens Manpower Form Modal | None | None | Renders workforce shift scheduling fields. |

| ****Manpower Modal Form**** | Fill fields: ****Employee Name**** (required), select ****Skill Level**** (High/Medium/Low), select ****Month****, ****Process/Trade**** (required), ****Man Hours**** (daily, default 8), ****Overtime**** (daily, default 0), ****Rate/Day (Tonnes)**** & click Save | Auto-calculates weekly manhours ($\text{[manhours + overtime]} * 5$) | `'POST /api/production/manpower/'` | Writes shift allocation data to `'Manpower'` table. | Closes modal, refreshes Manpower list. |

Module: Directory Masters (Employee, Customer, Detailer)

Manages profiles and capacities of employees, clients, and external detailing partners.

* ****Database Models****: `'employees.models.Employee'`, `'projects.models.Customer'`, `'projects.models.Detailer'`

* ****API Endpoints****:

* Employee: `'/api/employees/'` (GET/POST/PUT/DELETE)

* Customer: `'/api/projects/customers/'` (GET/POST/PUT/DELETE)

* Detailer: `'/api/projects/detailers/'` (GET/POST/PUT/DELETE)

UI Action → Result Mapping

| UI Element | User Action | Frontend Event | Backend API Call | DB Reads/Writes & Logic | Frontend Result |

| :--- | :--- | :--- | :--- | :--- | :--- |

| ****Add Employee Button**** | Click button | Resets form state, opens modal | None | None | Renders Employee modal dialog, active tab set to "Personal Profile". |

| ****Employee Modal Form (Personal Profile Tab)**** | Enter inputs: ****Full Name****, ****Email Address****, ****Phone Number**** (10-digit validation), ****Location****, ****Date of Birth****, ****Gender**** | Validates standard formatting, clicks "Next Step" | None | None | Modal transitions screen to display the "Professional Info" tab. |

| ****Employee Modal Form (Professional Info Tab)**** | Enter inputs: ****Employee ID**** (SFE-XXX), ****Designation****, select ****Department**** (Fabrication/Design/Quality/Admin/Operations), select ****Current Status**** (Active/On Leave/Inactive) | Checks required inputs, clicks "Create Identity" | `'POST /api/employees/'` | Writes profile metadata to the `'employees'` database table. | Closes modal dialog, triggers list refresh toast. |

| ****Delete Employee Icon**** | Click Trash icon | Prompts delete alert | `'DELETE /api/employees/<id>/'` | Deletes the `'Employee'` record from database. | Reloads Employee table. |

| ****Add Customer Form**** | Fill fields and click Save | Sends customer form data | `'POST /api/projects/customers/'` | Writes client company details to `'Customer'` table. | Closes modal, refreshes Customer Master grid. |

| ****Add Detailer Form**** | Fill fields and click Save | Sends detailer form data | `POST /api/projects/detailers/` | Writes detailer capability metadata to `Detailer` table. | Closes modal, refreshes Detailer Master grid. |

6. Sequence-Style Explanations of Critical Workflows

Workflow: RFQ Ingestion & Detail Allocation

```

| [User Interface]               | [Django API View]              | [Database] | [Filesystem] |
|--------------------------------|--------------------------------|------------|--------------|
|                                |                                |            |              |
| --- Upload Excel file ----->   |                                |            |              |
| (POST /api/rfq/upload/)        |                                |            |              |
|                                | --- Save temporary file -----> |            |              |
|                                | <-- Return file path -----     |            |              |
|                                |                                |            |              |
|                                | --- Read Excel rows ---->      |            |              |
|                                | (openpyxl parser)              |            |              |
|                                |                                |            |              |
|                                | --- Save RFQ records --->      |            |              |
|                                |                                |            |              |
| <-- Returns Ingested Count --- |                                |            |              |

```

Workflow: Local RAG Document Ingestion & Querying

```

| [User Interface]               | [Django Chatbot App]                  | [Ollama Local API] |
|--------------------------------|---------------------------------------|--------------------|
| [Database]                     |                                       |                    |
|                                |                                       |                    |
| --- Upload PDF File ----->     |                                       |                    |
| (POST /api/chatbot/upload)     |                                       |                    |
|                                | --- Parse text (pypdf) ----->         |                    |
|                                | --- Chunk sentences ----->            |                    |
|                                | --- Write context chunks to DB -----> |                    |
|                                |                                       |                    |
|                                | <-- Return chunk count -----          |                    |
| <-- Return ready status -----  |                                       |                    |
|                                |                                       |                    |
| --- Query chat: "How to..." -> |                                       |                    |
| (POST /api/chatbot/chat/)      |                                       |                    |
|                                | --- Search relevant chunks ----->     |                    |

```

	(BM25 ranking)	
	<-- Return top 3 chunks ----	
	--- Post query + context ->	
	(timeout = 180s)	
	<-- Return AI response -----	
	--- Write chat history ----->	
<-- Response + citations -----		
'''

```

---

## ## 7. File Upload, Download & Reporting Flows

### ### Excel RFQ Import/Export System

- \* **\*\*Parser\*\***: Managed in backend by `openpyxl`.
- \* **\*\*Action Flow\*\***:
  1. Estimator clicks **\*\*"Import Excel"\*\*** on `/api/rfq/`.
  2. Uploads `RFQ\_Master.xlsx`.
  3. Backend processes columns: `Quote No`, `Project Name`, `Structural Weight (Tons)`.
  4. Database updates existing rows matching `Quote No`, or creates new entries.

### ### PDF Report Exports

- \* **\*\*Action Flow\*\***:
  1. User clicks **\*\*"Print Proposal"\*\*** on `/steel-budget/result`.
  2. Frontend triggers browser `window.print()` using customized CSS print media sheets (`@media print`).
  3. Formats tables, hides navigation sidebars/headers, and generates a print/PDF export file.

---

## ## 8. Audit Logs & System Monitoring

### ### Request Logging Middleware

All incoming network requests to `/api/\*` are logged by `sfe\_project.middleware.RequestLoggingMiddleware`.

- \* **\*\*Log Destination\*\***: `backend/request\_log.txt`.
- \* **\*\*Metadata Logged\*\***:
  - \* Timestamp.
  - \* Target Path and HTTP Method.

- \* Authentication Authorization headers.
- \* Response status code and content preview snippet.

---

## ## 9. Error Handling & Validation Failures

The application handles network and model validation errors gracefully.

| Error Scenario                | Detection Level     | UI Message Displayed                                                 | System Action                                     |
|-------------------------------|---------------------|----------------------------------------------------------------------|---------------------------------------------------|
| ***Missing Fields**           | Frontend Form       | "This field is required" (Red border)                                | Blocks API transmission.                          |
| ***Ollama Server Offline**    | Backend View        | "Unable to connect to local Ollama server. Check if it's running..." | Logs error to `request_log.txt`, returns 500.     |
| ***Timeout (CPU overloaded)** | Network Client      | "Chatbot Error: timed out"                                           | Aborts request after 180s, displays retry button. |
| ***Invalid JWT Token**        | Axios Interceptor   | Redirects to `/login`                                                | Clears local `sessionStorage` token parameters.   |
| ***Duplicate RFQ ID**         | Database Constraint | "RFQ with this Quote Number already exists"                          | Triggers DB rollback, returns 400.                |

---

## ## 10. Glossary of Technical Terms

- \* **BM25**: A keyword-based retrieval algorithm that calculates relevance scores matching queries to document chunks.
- \* **Detailer**: External engineering group responsible for drafting detail drawings for the fabrication shop.
- \* **Erection**: The process of hoisting and bolting steel members together in the field using cranes.
- \* **In-Context Learning (RAG)**: Embedding document pages inside an LLM's prompt window to provide factual reference context without modifying model weights.
- \* **Ollama**: A lightweight local model runner used to deploy LLMs (like Llama) on local development hardware offline.
- \* **RFQ**: Request for Quotation; the initial client request containing structural drawing documents.
- \* **RTS Date**: Release to Shop date; the final date when detailing drawings are approved and sent to shop fabrication.
- \* **VRAM**: Video RAM; dedicated graphics card memory necessary to run Ollama offline models smoothly.